



Reconnaissance faciale

MATLAB

Rayan Kobrossly



23/10/2024



Sommaire

INTRODUCTION.....	3
I - Base d'images.....	3
II - Calcul visage moyen.....	4
III - Calcul des caractéristiques d'un visage.....	5
IV - Projection dans le sous espace des visages.....	6
V - Identification d'un visage.....	7
VI - Classification visage/non visage.....	10
CONCLUSION.....	13
CODE.....	13



INTRODUCTION

Dans ce compte rendu, on s'intéresse à déterminer un moyen d'implémenter la reconnaissance faciale sur Matlab. La reconnaissance faciale consiste à capturer des images d'entraînement pour la machine, les convertir en données, puis déduire ses caractéristiques. A la fin, on compare les images inconnues à la base de données pour bien identifier la personne.

Ce travail requiert des notions de calculs matriciels comme les projections sur d'autres bases, diagonalisation et multiplications matricielles.

On commence par traiter des vecteurs qui définissent une image, on calcule un visage centré, une reconstruction du visage, une projection sur une autre base et finalement une notion de distance entre une image et la base de données pour attribuer les identités spécifiques.

Il faudra noter que durant l'exploitation des résultats des différentes parties, le code commenté ne sera pas toujours présent pour des raisons de visibilité, mais il sera inclus à la fin comme la version html de Matlab.

I - Base d'images

Une image est une matrice de données, plus généralement une matrice carrée. Pour faciliter nos calculs, on convertit une image en un vecteur colonne. Alors une image définie par un vecteur 4096x1 est une image 64x64. En plus, dans notre application spécifique, on traite que des images noir et blanc, alors les valeurs sont comprises entre 0 et 255. Il est intéressant de noter qu'il est essentiel d'arrondir les données à notre plage de nombres. Ceci est fait en utilisant la fonction `mat2gray` pré-définie sur Matlab.

Et pour afficher l'image sur un plot, il faudra utiliser `imshow` qui convertit une matrice «arrondie» en une image.

Pour l'afficher, il faudra reconvertir la matrice 4096x1 en 64x64. On crée notre fonction `imagereto64` qui prend en entrée le vecteur colonne et renvoie la matrice carrée, voici son code :

```
function matrix_square = imagereto64(matrix_ligne)

matrix_square = zeros(64,64);
if size(matrix_ligne)~= [4096,1]

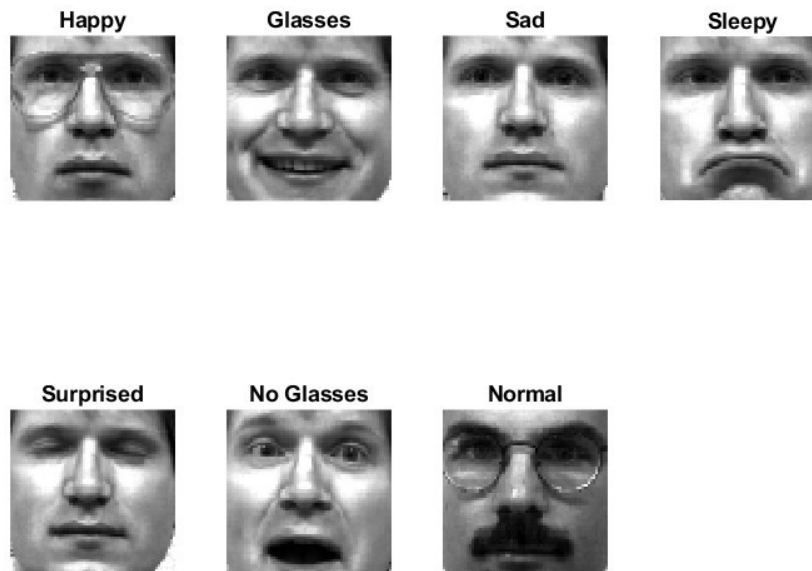
fprintf('Error, matrix size doesnt match\n');
else
    n=1;
    for k=1:4096
        if mod(k,64)~=1 && k~=1
            n=n+1;
        end
        j = k - (n-1)*64;
        matrix_square(j,n) = matrix_ligne(k);
    end
end
```

Dans mon cas, j'avais utilisé cette méthode de modulo, ou reste de division pour savoir quand nous avons compté 64 éléments. On pouvait bien-sûr utiliser autre méthode, mais j'ai fait comme ça, une façon similaire au langage de programmation C.



Dans les fichiers de données que nous avons importés, il existe 2 matrices importantes X_{train} et X_{test} qui contiennent respectivement 90 et 30 photos.

Pour afficher 7 photos, il suffit de convertir les vecteurs colonnes en matrices carrées et arrondir les valeurs à des valeurs entre 0 et 255, et utiliser la fonction subplot. Voici le résultat attendu :



Les titres ne sont pas très adaptés par contre.

II - Calcul visage moyen

Le calcul moyen sert à déterminer une façon pour comparer tous les visages. On crée une version centrée de l'image. Pour calculer ce vecteur colonne moyen, on utilise la fonction mean de Matlab.

Maintenant, on le visage moyen x_{moy} de dimension 4096x1. On le soustrait que chaque vecteur colonne de X_{train} pour obtenir une matrice de visages centrés X_c .

On affiche le visage centré, 2 images de X_{train} à côté de leurs correspondante de la matrice centrée, on obtient le résultat suivant:





On remarque que les visages centrés sont «déformés».

III - Calcul des caractéristiques d'un visage

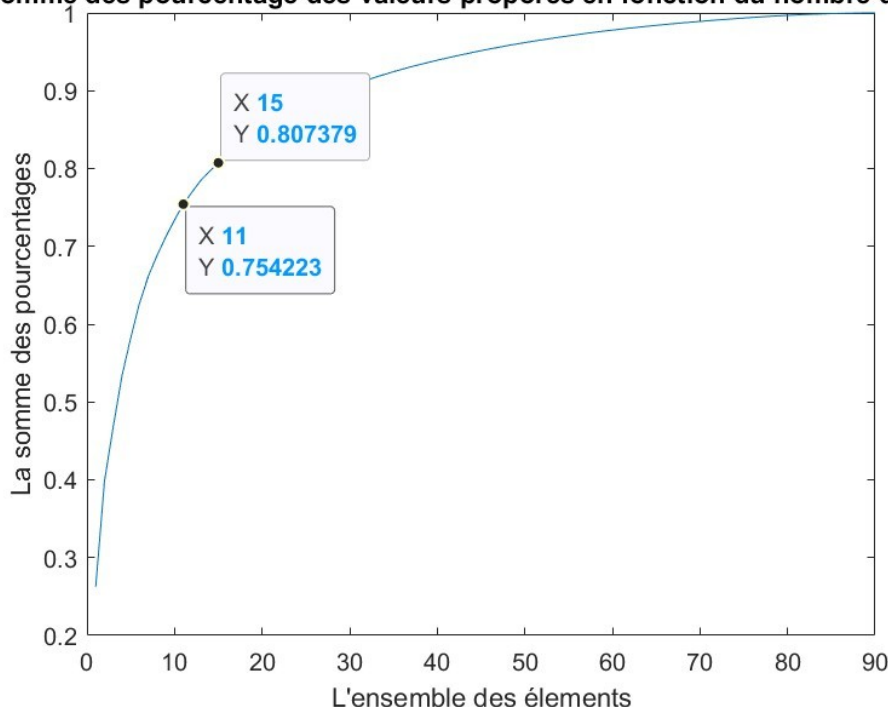
Dans cette partie, pour pouvoir calculer la ressemblance entre les images, il faudra diagonaliser la matrice X_c 4096x90. Elle sera définie sous cette forme $X_c = U * S * V^T$ où U et V sont des matrices orthogonales (le produit avec leurs matrices transposées donne l'identité) et S une matrice diagonale qui contient la racine carrée des valeurs propres (principe de la diagonalisation).

L'écriture du code de la diagonalisation peut avérer difficile. Heureusement, Matlab traite les calculs sur les matrices, et il possède une fonction prédéfinie svd tel que $[U, S, V] = \text{svd}(X_c, 0)$

On récupère les matrices U, S et V. On remarque, si on multiplie U par sa transposée, soustrait la matrice identité et on calcule la norme, on obtient une valeur de 4×10^{-15} ce qui est très proche de 0.

On calcule la matrice S^2 , on fait bien attention à utiliser $.^2$ pour faire le carré des termes et non pas calculer un produit matriciel, et on utilise $\text{diag}(S)$ pour transformer S en un vecteur colonne. Les valeurs ainsi obtenues sont appelées valeurs propres et donnent l'importance du vecteur caractéristique. Plus la valeur propre est élevée, plus le vecteur caractéristique associé apportera des informations sur le visage. La somme de toutes ces valeurs est égale à 6.4×10^8 . On calcule le pourcentage de chaque valeur propre par rapport à la somme de ces valeurs, on évalue la contribution de chaque valeur. (C'est un vecteur 90x1) On calcule sum_p_S_carre qui est un vecteur dont la ligne i est la somme des lignes précédentes et de son pourcentage. sum_p_S_carre est alors fonction de k, le nombre de ligne. Cette fonction monotone croissante tend vers 1 pour $k = 90$. Voici cette courbe :

La somme des pourcentages des valeurs propres en fonction du nombre d'éléments



On trouve pour $k=15$, nous avons une reconstruction de 80%. On arrive à une reconstruction de 75% pour $k = 11$, ce qui est impressionnant avec le nombre de valeurs qu'on a.



IV - Projection dans le sous espace des visages

Après avoir calculé les valeurs propres, il faudra projeter le visage dans une nouvelle base, qui est la base formée par les vecteurs caractéristiques.

Un visage alors, possède sa représentation en un vecteur colonne et ses coordonnées dans la nouvelle base. Elles sont calculées en utilisant cette formule $z = U_K^T * (x - x_{moy})$ ou $z = U_K^T * X_c(:, i)$

où U_K n'as que les K premières colonnes (k premiers visages). $X_c(:, i)$ représente un vecteur colonne 4096x1, c'est le vecteur d'un visage x donné. z est de dimension kx1.

La reconstruction du visage x à l'aide des vecteurs caractéristique est donnée par la formule suivante

$$\hat{x} = x_{moy} + U_K * z$$

Nous souhaitons calculer la reconstruction de n'importe quel visage x. On choisi arbitrairement $x = 1$.

Nous calculons z pour $k = 90$, on prend alors toutes les colonnes de U. z est donc de taille 90x1, puis la reconstruction $\hat{x}$ de x.

L'erreur entre x et $\hat{x}$, qui sont tous les deux des vecteurs 4096x1, est la norme 2 entre ces 2 valeurs

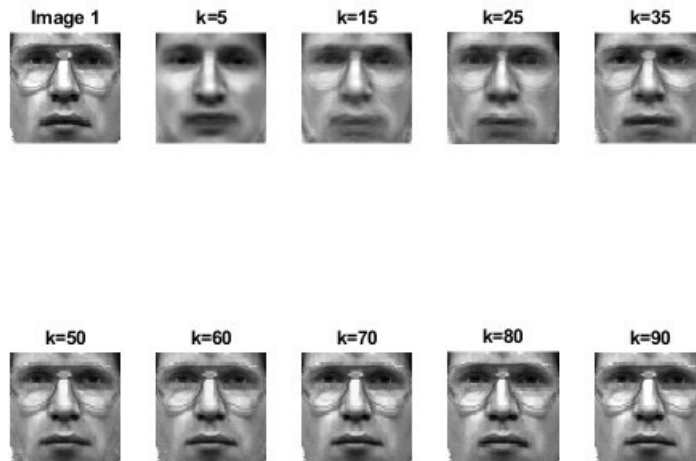
$e = \|x - \hat{x}\|_2$. Pour l'image 1 et pour $k = 90$, on trouve $e = 0$, ce qui est logique vu qu'on prend toutes les lignes. Mais dès qu'on diminue k, on constate que l'image reconstruite devient moins nette et l'erreur entre elle et l'originale augmente et atteint 1827 pour $k=5$.

```
error is equal to 1827.7191620103
error is equal to 1217.3142683121
error is equal to 1050.0511038043
error is equal to 807.4367721381
error is equal to 373.6548213288
error is equal to 255.8641123311
error is equal to 107.6383122866
error is equal to 53.1823807393
error is equal to 0.0000000000
```

C'est pour les valeurs de k dans la liste suivante :

liste_k={5;15;25;35;50;60;70;80;90};

La reconstruction des visages du visage $x = 1$ pour les différentes valeurs de k donne le résultat suivant.



On trouve bien que dès $k=15$, on atteint une netteté quasi totale (80,7%). On pourrait répéter le même processus pour un visage de X_{test} . Nous obtiendrions un résultat similaire.

V - Identification d'un visage

A chaque visage de la base X_{train} , on associe un identifiant spécifique qui dépend du visage (ou bien la personne)

Pour identifier un visage de la base inconnue X_{test} , il faut introduire une notion de distance $E_k(x_{test})$ tel que $E_k(x_{test}) = \|z^{test} - z_k^{train}\|_2$. On voit que cette distance est la différence entre l'image qu'on souhaite identifier, et chaque valeur de X_{train} . Donc, la personne de la base d'entraînement dont sa projection a la distance minimale, est donc la personne de la base de test. On peut confirmer l'identité. $E_k(x_{test})$ est donc de taille 1×90 si on prend toutes les visages.

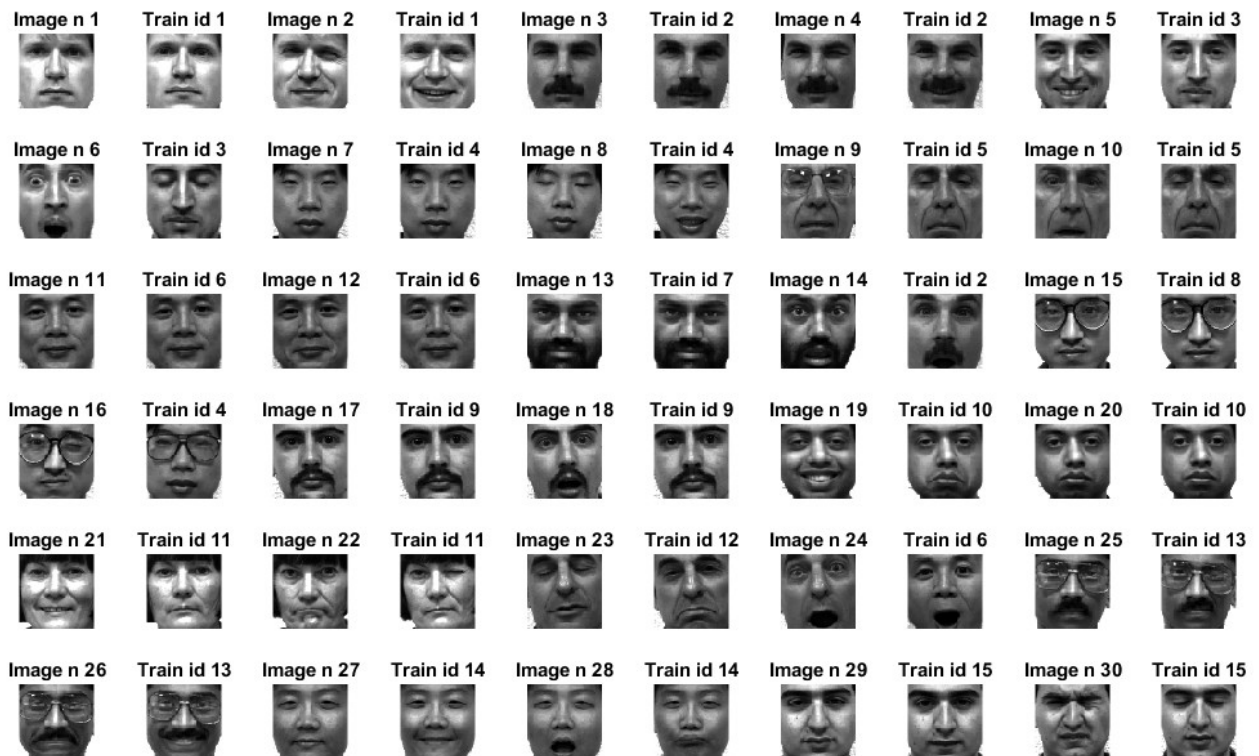
z^{test} est calculé de la même façon que $z_{train}(x)$.

Dans un premier cas, pour $k = 90$, c'est à dire $U_k = U$, images nettes, pour toutes les images de test (une boucle qui se répète 30 fois, une fois par image), on calcul le vecteur distance, on trouve son minimum et son indice associé. Si maintenant le choisi la valeur du vecteur d'identité avec le même indice, on trouve l'identité de la personne inconnue.

Dans ce cas, voici les identités pour $k=90$:



For $k = 90$



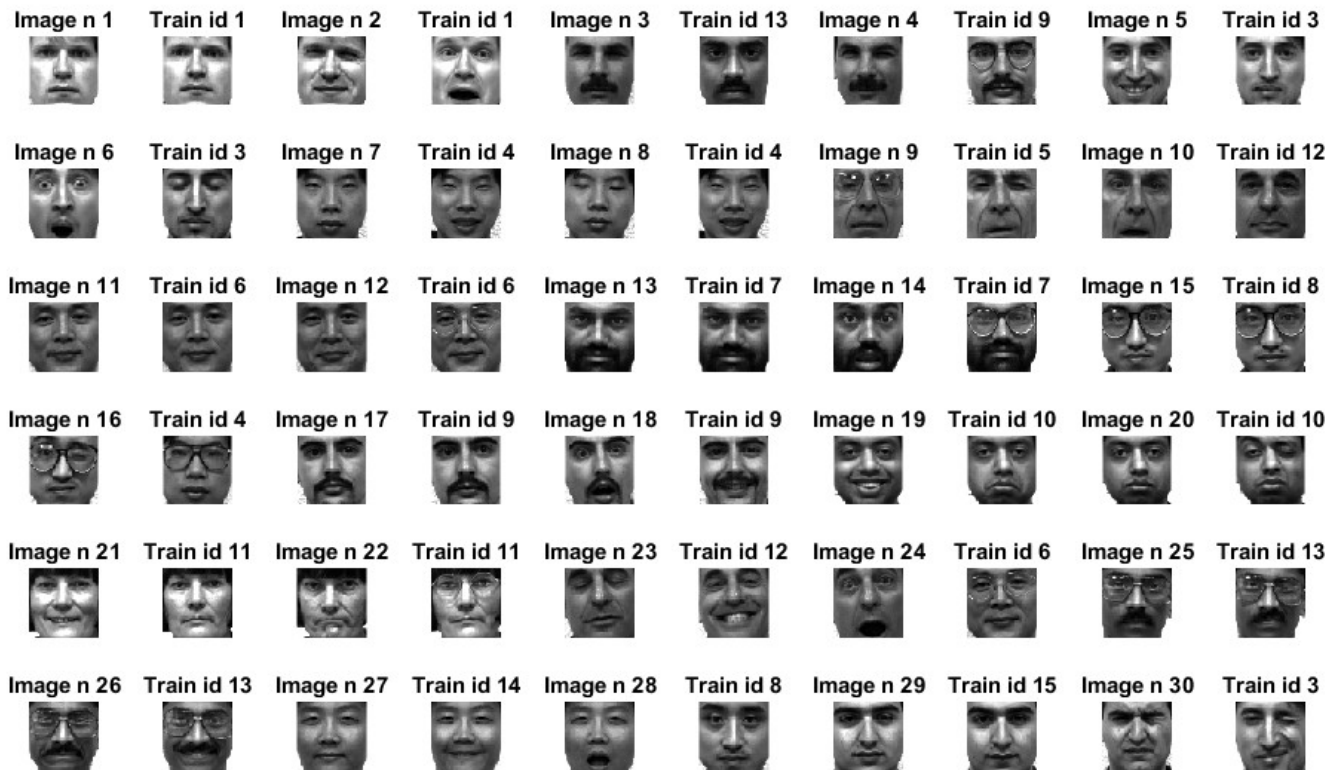
On trouve même pour la meilleure netteté, nous avons 3 personnes mal identifiées, c'est les personnes 14, 16 et 24. Une précision de 90%

Maintenant, pour mes valeurs de k suivantes $\text{liste_k}=\{5;15;30;35;50;60;70;80;90\}$, il suffit que mettre tout ce qu'on a fait dans une grande boucle for, qui répète cette démarche pour les différentes valeurs de k de la liste.

La taille du subplot n'est donc pas modifiée. Voici pour $k = 5$:



For $k = 5$

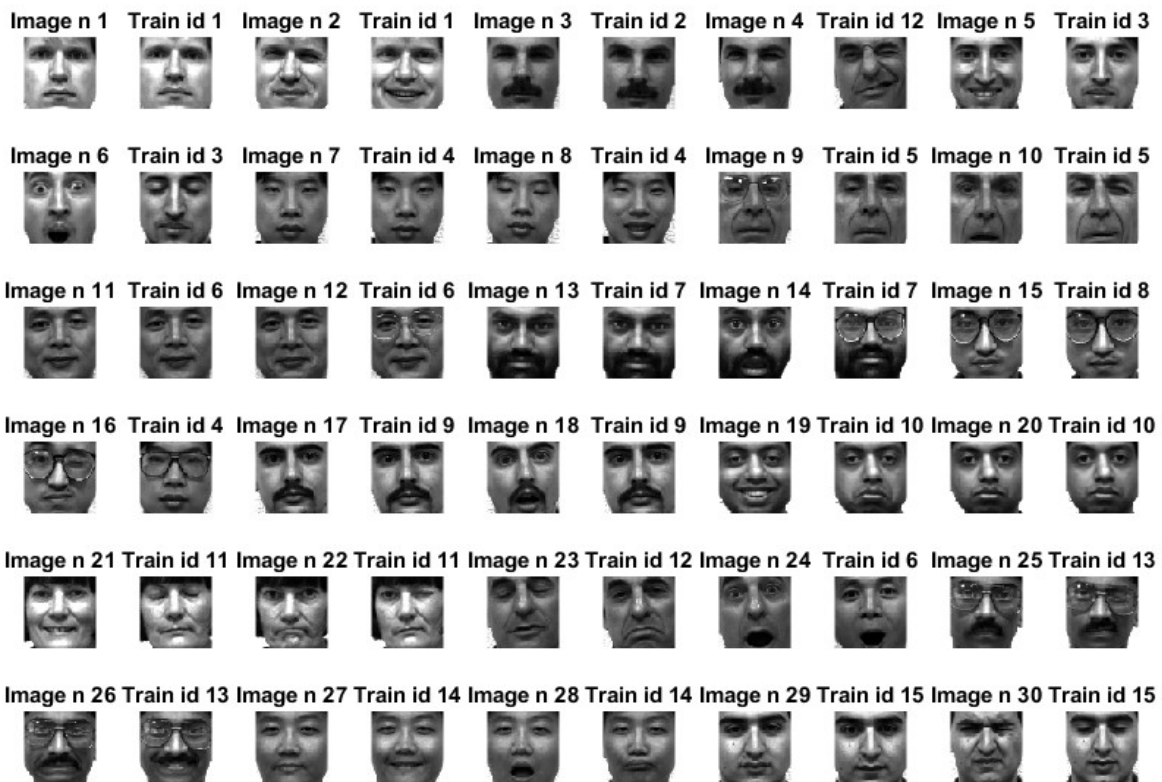


Les personnes mal identifiées sont 16,3,28,4,24. Une précision de 84%Ce qui est très bien vu que la netteté de la reconstruction des images est à 50% quasiment.

Un dernier exemple pour $k = 30$:



For $k = 30$



Les seules personnes mal identifiées sont les même pour $k=90$, vu que la netteté dans le graphe présenté avant était de 90%

Pour tous les autres valeurs de k supérieurs à 30, nous avons donc le même résultat. Juste l'erreur est plus petite. (Les autres images sont avec le code à la fin du pdf)

VI - Classification visage/non visage

Dans la partie précédente, nous avons identifié la personne de la base de test. Maintenant, nous utiliserons des notions similaires pour savoir si une image est un visage ou non.

Nous avons une base d'objets (visages et non visages) X_{noface}

Pour savoir identifier les objets, il faudra trouver un algorithme qui prend certain paramètres en compte et renvoie oui ou non pour la question est-ce un visage?

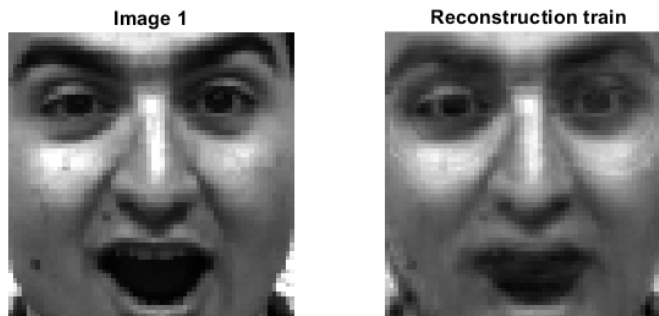
Pour faire ceci, il faudra déterminer les erreurs de reconstruction des images des bases d'entraînement et de tests. On choisi $k=30$, netteté de 90%, on calcul les maximums de ces erreurs (un visage a au plus, dans notre base de donnée, telle erreur de reconstruction).

Pour la base train, l'erreur maximum est 1080, une valeur moyenne de 811. Pour la base de test, l'erreur maximum est de 1730 avec une moyenne de 1200.

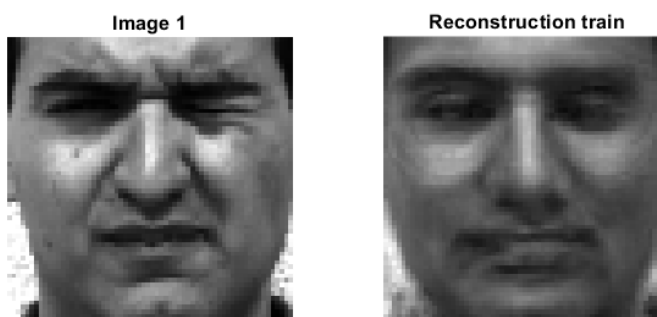


On a choisi à représenter les dernières image reconstruites de train et test, voici-les:

Dans la base de données d'entraînement:



Et pour X_{test} :



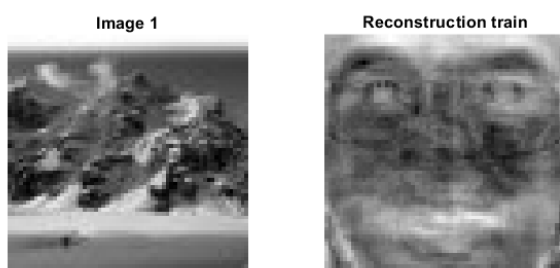
Les reconstructions sont très similaires. Maintenant, nous avons une idée des erreurs.

Maintenant, il faudra calculer l'erreur de reconstruction de X_{noface} . Logiquement, pour qu'un objet soit un visage, son erreur de reconstruction doit être inférieure à la valeur maximale d'erreur de reconstruction de test de d'entraînement.

Le minimum de X_{noface} est de 1500 avec une valeur moyenne de 2600.

Comme attendu, vu qu'on a des images d'objets qui sont pas des visages, la valeur moyenne est trop élevée.

Voici la représentation d'une image de X_{noface}



La photo de la montagne reconstruite ne ressemble ni à un visage ni une montagne.

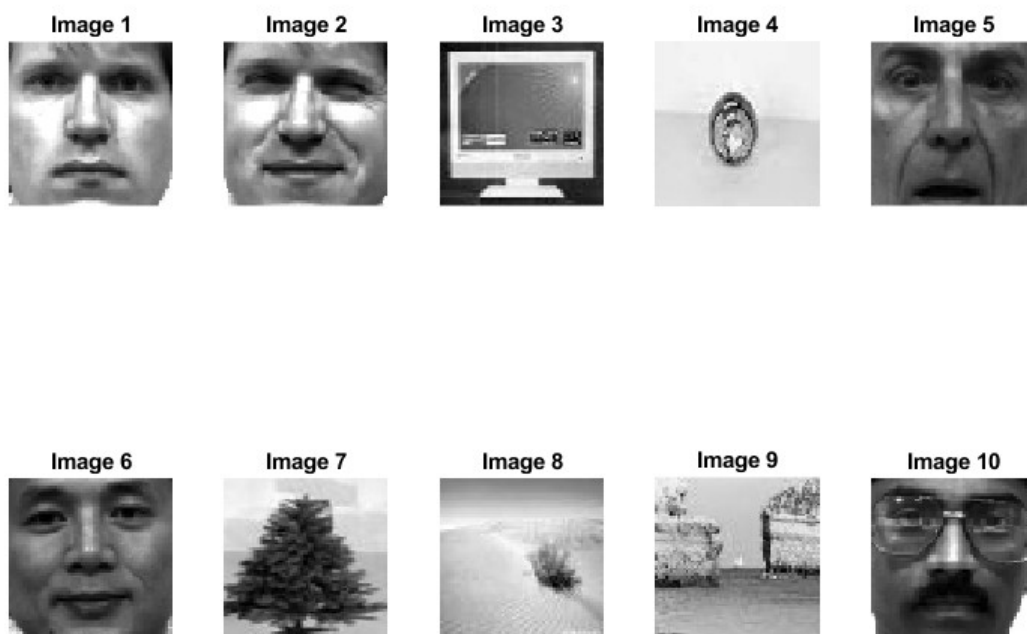


Maintenant, X_{noface} contient 10 images, on propose un algorithme qui prend en compte l'erreur de reconstruction d'un objet et renvoie si c'est un visage ou non. Voici notre formule de calcul de minimum d'erreur d'un visage :

$$\text{erreur}_{\text{face}} = \frac{\max_{\text{train}} + \max_{\text{test}}}{2} + \frac{\text{moy}_{\text{train}} + \text{moy}_{\text{test}}}{4} \text{ elle vaut 1844 dans notre cas pour } k=30$$

Si l'erreur de reconstruction est inférieur à cette valeur, c'est un visage, sinon c'est pas un visage.

Voici les images de X_{noface} :



Et voici les résultat de notre algorithme :

```
Voici les images qu'on doit classifier
the image avec le numero 1 est un visage
the image avec le numero 2 est un visage
the image avec le numero 3 n est pas un visage
the image avec le numero 4 n est pas un visage
the image avec le numero 5 est un visage
the image avec le numero 6 est un visage
the image avec le numero 7 n est pas un visage
the image avec le numero 8 est un visage
the image avec le numero 9 n est pas un visage
the image avec le numero 10 est un visage
```



Cet algorithme a une précision de 90%. L'image 8 avait une erreur de 1400 ce qui est très similaire aux valeurs d'une image d'un visage normal.

CONCLUSION

Nous avons donc pu appliquer les notions de reconnaissance faciale, nous avons bien identifier les personnes avec une précision entre 84% et 90%. Nous avons encore, avec une précision de 90%, déterminer si un objet était un visage ou non en calculant des erreurs de reconstruction.

Pour améliorer nos algorithmes, c'était meilleur d'avoir beaucoup plus d'images d'entraînement, disons 1000 images, les valeurs des erreurs seront plus petites et nous pourrons optimiser plus l'algorithme de visage/pas visage

CODE

Le code Matlab sera affiché sur la page suivante. Il est commenté, divisé en sous-parties et il inclut les représentations des plots et subplots.

Contents

- [RECONAISSANCE FACIALE](#)
- [Informations générales](#)
- [Rendre le vecteur colonne en une matrice 64x64](#)
- [Représentation des images 64x64](#)
- [Calcul du visage moyen](#)
- [Calcul des caractéristiques d'un visage](#)
- [Projection dans le sous espace des visages](#)
- [Identification d'un visage](#)
- [Pour aller plus loin](#)

RECONAISSANCE FACIALE

Informations générales

```
% Auteur : Rayan Kobrossly
% TD7 IA2R
% -----

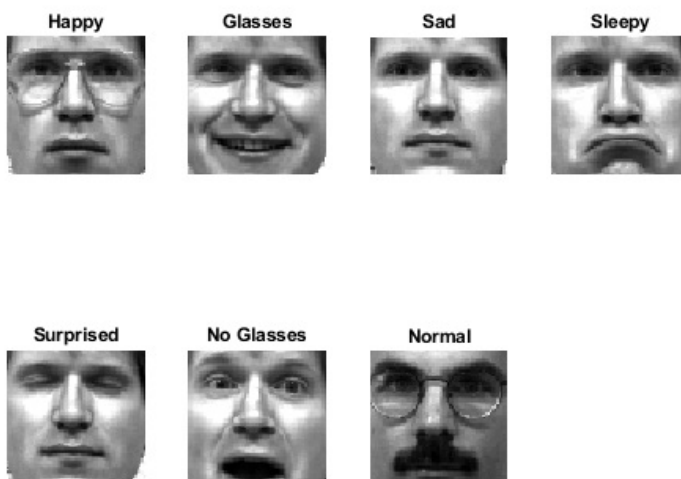
clear
clc
close all;
load Yalefaces.mat
load baseunknown.mat
% size_Xtrain = size(X_train);
```

Rendre le vecteur colonne en une matrice 64x64

```
for i=1:90
    cat_train(i) = mat2gray(imageto64(X_train(:,i))); %le fait en une seule ligne de code
    % uint8 et mat2gray transforme le double en un entier positif entre 0
    % et 255
end
```

Représentation des images 64x64

```
figure;
titres = {'Happy', 'Glasses', 'Sad', 'Sleepy', 'Surprised', 'No Glasses', 'Normal'};
for i=1:7
    subplot(2,4, i);
    imshow(cat_train(i));
    title(titres{i});
end
```



Calcul du visage moyen

```
xmoy = mean(X_train,2); %calcul le visage moyen de toutes les 90 photos
centre = mat2gray(imageto64(xmoy)); %%transforme le vecteur colonne 4096x1 en 64x64
figure;
plot(centre);
imshow(centre);

X_c = X_train - xmoy; %calcul image centrée

figure(10);
subplot(1,5,1); % affiche visage centre
imshow(centre);
title("Image visage centrée");

subplot(1,5,2); % affiche visage n1 non centrée
imshow(cat_train{1});
title("Image n1 non centrée");

subplot(1,5,3); % affiche visage n1 centrée
imshow(uint8(imageto64(X_c(:,1))));
title("Image n1 centrée");

subplot(1,5,4); % affiche visage n1 non centrée
imshow(cat_train{50});
title("Image n2 non centrée");

subplot(1,5,5); % affiche visage n1 centrée
imshow(uint8(imageto64(X_c(:,50))));
title("Image n2 centrée");
```

Image visage centrée Image n1 non centrée Image n1 centrée Image n2 non centrée Image n2 centrée



Calcul des caractéristiques d'un visage

```
[U, S, V] = svd(X_c, 0);
orthonormality_U = norm(U' * U - eye(size(U, 2))); % Devrait être quasi égale à 0

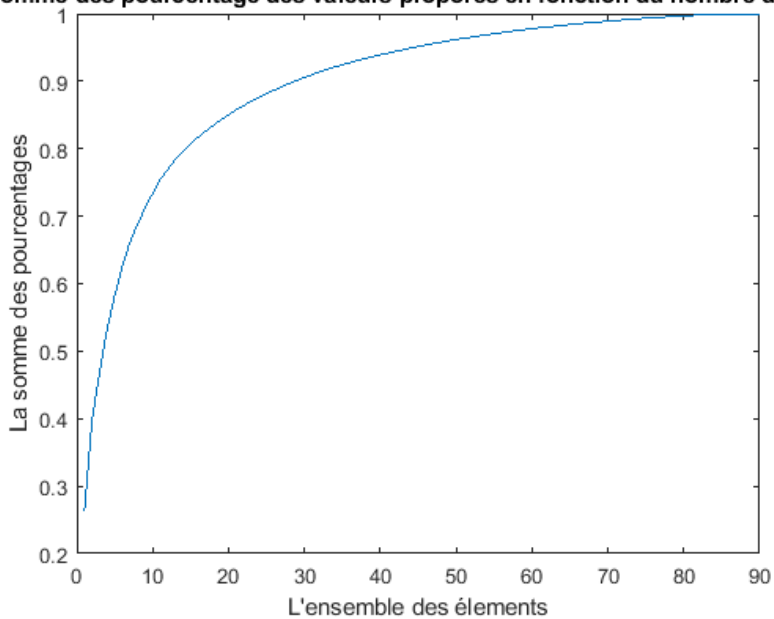
S_carre = diag(S).^2; % calcul le carre de chaque valeur, la matrice diagonale est mise en un vecteur colonne
sum_S_carre = sum(S_carre);
p_S_carre = S_carre/sum_S_carre; % pourcentage de valeurs propres

sum_p_S_carre = zeros(90,1);
sum_p_S_carre(1,1) = p_S_carre(1,1);
for k=2:90
    sum_p_S_carre(k,1) = sum_p_S_carre(k-1,1) + p_S_carre(k,1);
end

v_k = (1:1:90)'; %pour tracer la courbe de somme des pourcentage des valeurs propres en fonction de nombre d'elements
figure(11);
plot(v_k,sum_p_S_carre);
title("La somme des pourcentage des valeurs propres en fonction du nombre d'éléments");
xlabel("L'ensemble des éléments");
```

```
ylabel("La somme des pourcentages");
% au bout de 15 images, le pourcentage des valeurs proposees est de 80.7%
% pour avoir 75% des données, il faut avoir recours à seulement 11 éléments
```

La somme des pourcentage des valeurs proposees en fonction du nombre d'éléme



Projection dans le sous espace des visages

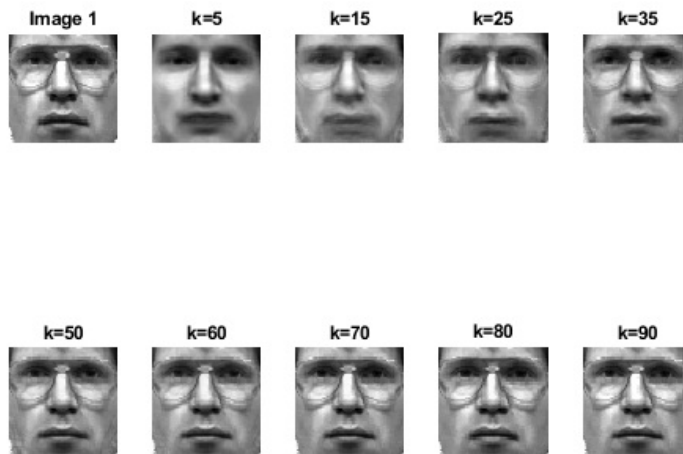
```
indice_x = 1;
j = 2;
liste_k = {5;15;25;35;50;60;70;80;90};
for i = 1:length(liste_k) %tester pour les differentes valeurs de k
    k = liste_k{i};
    x = X_train(:,indice_x); %prenons comme exemple la premiere image
    z = U(:,1:k)' * X_c(:,indice_x); %x-xmoyenne represente l'image centree de x
    % il y a pas le . avant * car c'est une multiplication matricielle non pas
    % terme à terme
    x_reconstruit = xmoy + U(:,1:k)*z; %l'image reconstruite de x
    %U(:,k).*z doit etre 4096x1 non pas 4096*4096

    e = norm(x-x_reconstruit); %l'erreur entre l'image et on image reconstruite
    fprintf('error is equal to %.10f\n', e);

    figure(11);
    subplot(2,5,1); % image origine
    imshow(mat2gray(imageto64(x)));
    title("Image 1");
    subplot(2,5,j); % image origine
    imshow(mat2gray(imageto64(x_reconstruit)));
    title("k="+k);
    j = j+1;

end
clear j
clear i
clear k
clear indice_x
```

```
error is equal to 1827.7191620103
error is equal to 1217.3142683121
error is equal to 1050.0511038043
error is equal to 807.4367721381
error is equal to 373.6548213288
error is equal to 255.8641123311
error is equal to 107.6383122866
error is equal to 53.1823807393
error is equal to 0.0000000000
```

Identification d'un visage

```

%*****
liste_k={5;30;50;75;90};
for ii=1:length(liste_k) %tester pour les differentes valeurs de k
    k = liste_k{ii};
    indice_subplot =1;

    for indice_test=1:30 %pour toutes les personnes de X_test

        e_k_distance = zeros(1, k); %on initialise la distance e_k_distance
        z_moy = mean(X_test,2);
        Z_c = X_test-z_moy;
        z_test = U(:,1:k)' * Z_c(:,indice_test); %l'image de la personne de la matrice test
        for i=1:90
            z_train = U(:,1:k)' * X_c(:,i); % l'un des 90 personnes de la base train pour pouvoir calculer la distance entre les personnes
            e_k_distance(i) = norm(z_test-z_train); %les differents elements du vecteur distance 1xk = 1x90
        end

        [min_e_k, indice_e_k] = min(e_k_distance); %calcul de la distance minimale= personne la plus probable d'etre bien identifiée
        fprintf('Minimum value: %.4f at index: %d\n', min_e_k, indice_e_k);
        indice_individu_test = id_train(indice_e_k);
        fprintf('the individual %d id is equal to %d\n',indice_test, indice_individu_test);

    %*****

    figure(12+ii) % on aura plusieurs figures pour les differentes valeurs de k
    subplot(6,10,indice_subplot); % image origine
    imshow(mat2gray(imageto64(X_test(:,indice_test)))); %trace l'image de X_test
    title("Image n "+indice_test);
    indice_subplot = indice_subplot+1;
    subplot(6,10,indice_subplot); % trace l'image de la personne la plus proche
    imshow(mat2gray(imageto64(X_train(:,indice_e_k))));
    title("Train id "+id_train(indice_e_k));
    indice_subplot = indice_subplot+1;

    sgtitle("For k = " + string(k));

end

end

```

```

Minimum value: 592.5630 at index: 3
the individual 1 id is equal to 1
Minimum value: 386.7989 at index: 6
the individual 2 id is equal to 1
Minimum value: 571.3319 at index: 75
the individual 3 id is equal to 13
Minimum value: 589.7498 at index: 49
the individual 4 id is equal to 9
Minimum value: 788.8072 at index: 15
the individual 5 id is equal to 3

```

Minimum value: 935.1097 at index: 17
the individual 6 id is equal to 3
Minimum value: 323.1163 at index: 20
the individual 7 id is equal to 4
Minimum value: 462.6017 at index: 20
the individual 8 id is equal to 4
Minimum value: 601.3460 at index: 30
the individual 9 id is equal to 5
Minimum value: 409.9810 at index: 69
the individual 10 id is equal to 12
Minimum value: 190.7636 at index: 33
the individual 11 id is equal to 6
Minimum value: 277.5361 at index: 31
the individual 12 id is equal to 6
Minimum value: 131.0248 at index: 39
the individual 13 id is equal to 7
Minimum value: 769.4487 at index: 37
the individual 14 id is equal to 7
Minimum value: 288.2536 at index: 45
the individual 15 id is equal to 8
Minimum value: 454.6464 at index: 19
the individual 16 id is equal to 4
Minimum value: 415.1527 at index: 51
the individual 17 id is equal to 9
Minimum value: 281.5595 at index: 50
the individual 18 id is equal to 9
Minimum value: 404.5173 at index: 57
the individual 19 id is equal to 10
Minimum value: 496.2144 at index: 60
the individual 20 id is equal to 10
Minimum value: 388.0359 at index: 63
the individual 21 id is equal to 11
Minimum value: 1186.6840 at index: 61
the individual 22 id is equal to 11
Minimum value: 395.1003 at index: 68
the individual 23 id is equal to 12
Minimum value: 581.3558 at index: 31
the individual 24 id is equal to 6
Minimum value: 383.2745 at index: 78
the individual 25 id is equal to 13
Minimum value: 229.7224 at index: 74
the individual 26 id is equal to 13
Minimum value: 725.4628 at index: 80
the individual 27 id is equal to 14
Minimum value: 641.3476 at index: 44
the individual 28 id is equal to 8
Minimum value: 485.2957 at index: 87
the individual 29 id is equal to 15
Minimum value: 746.0171 at index: 18
the individual 30 id is equal to 3
Minimum value: 1176.1551 at index: 3
the individual 1 id is equal to 1
Minimum value: 1230.8262 at index: 2
the individual 2 id is equal to 1
Minimum value: 1052.1626 at index: 10
the individual 3 id is equal to 2
Minimum value: 1853.8206 at index: 72
the individual 4 id is equal to 12
Minimum value: 1383.2987 at index: 15
the individual 5 id is equal to 3
Minimum value: 1870.3917 at index: 17
the individual 6 id is equal to 3
Minimum value: 623.0179 at index: 22
the individual 7 id is equal to 4
Minimum value: 1362.4016 at index: 20
the individual 8 id is equal to 4
Minimum value: 1089.6713 at index: 26
the individual 9 id is equal to 5
Minimum value: 948.8467 at index: 30
the individual 10 id is equal to 5
Minimum value: 378.1420 at index: 33
the individual 11 id is equal to 6
Minimum value: 568.8409 at index: 31
the individual 12 id is equal to 6
Minimum value: 365.2845 at index: 39
the individual 13 id is equal to 7
Minimum value: 2073.3645 at index: 37
the individual 14 id is equal to 7
Minimum value: 852.7015 at index: 45
the individual 15 id is equal to 8
Minimum value: 1676.5356 at index: 19
the individual 16 id is equal to 4
Minimum value: 986.5564 at index: 51

the individual 17 id is equal to 9
Minimum value: 1448.3703 at index: 51
the individual 18 id is equal to 9
Minimum value: 1761.3469 at index: 57
the individual 19 id is equal to 10
Minimum value: 882.8574 at index: 56
the individual 20 id is equal to 10
Minimum value: 1209.3292 at index: 64
the individual 21 id is equal to 11
Minimum value: 1576.9531 at index: 66
the individual 22 id is equal to 11
Minimum value: 1173.0656 at index: 71
the individual 23 id is equal to 12
Minimum value: 1381.6027 at index: 35
the individual 24 id is equal to 6
Minimum value: 934.0950 at index: 78
the individual 25 id is equal to 13
Minimum value: 1013.2931 at index: 74
the individual 26 id is equal to 13
Minimum value: 1033.7696 at index: 80
the individual 27 id is equal to 14
Minimum value: 1377.7418 at index: 82
the individual 28 id is equal to 14
Minimum value: 1294.0061 at index: 87
the individual 29 id is equal to 15
Minimum value: 1918.8507 at index: 89
the individual 30 id is equal to 15
Minimum value: 1278.4689 at index: 3
the individual 1 id is equal to 1
Minimum value: 1525.4861 at index: 2
the individual 2 id is equal to 1
Minimum value: 1183.6199 at index: 9
the individual 3 id is equal to 2
Minimum value: 1951.8188 at index: 8
the individual 4 id is equal to 2
Minimum value: 1461.1963 at index: 15
the individual 5 id is equal to 3
Minimum value: 2052.0189 at index: 17
the individual 6 id is equal to 3
Minimum value: 760.2790 at index: 22
the individual 7 id is equal to 4
Minimum value: 1501.0664 at index: 20
the individual 8 id is equal to 4
Minimum value: 1183.7111 at index: 28
the individual 9 id is equal to 5
Minimum value: 1081.7557 at index: 30
the individual 10 id is equal to 5
Minimum value: 438.8545 at index: 33
the individual 11 id is equal to 6
Minimum value: 612.0916 at index: 31
the individual 12 id is equal to 6
Minimum value: 413.8725 at index: 39
the individual 13 id is equal to 7
Minimum value: 2200.2046 at index: 12
the individual 14 id is equal to 2
Minimum value: 1054.5598 at index: 45
the individual 15 id is equal to 8
Minimum value: 2002.6447 at index: 19
the individual 16 id is equal to 4
Minimum value: 1056.0014 at index: 51
the individual 17 id is equal to 9
Minimum value: 1620.0184 at index: 51
the individual 18 id is equal to 9
Minimum value: 1853.8350 at index: 57
the individual 19 id is equal to 10
Minimum value: 979.3932 at index: 56
the individual 20 id is equal to 10
Minimum value: 1371.6310 at index: 63
the individual 21 id is equal to 11
Minimum value: 1711.0298 at index: 66
the individual 22 id is equal to 11
Minimum value: 1339.2728 at index: 71
the individual 23 id is equal to 12
Minimum value: 1462.3960 at index: 35
the individual 24 id is equal to 6
Minimum value: 1215.5225 at index: 78
the individual 25 id is equal to 13
Minimum value: 1174.5150 at index: 74
the individual 26 id is equal to 13
Minimum value: 1097.5862 at index: 80
the individual 27 id is equal to 14
Minimum value: 1442.5039 at index: 82
the individual 28 id is equal to 14

Minimum value: 1483.9210 at index: 87
the individual 29 id is equal to 15
Minimum value: 2116.7790 at index: 89
the individual 30 id is equal to 15
Minimum value: 1369.5522 at index: 3
the individual 1 id is equal to 1
Minimum value: 1627.0523 at index: 2
the individual 2 id is equal to 1
Minimum value: 1286.8139 at index: 9
the individual 3 id is equal to 2
Minimum value: 2022.6771 at index: 8
the individual 4 id is equal to 2
Minimum value: 1511.6569 at index: 15
the individual 5 id is equal to 3
Minimum value: 2144.7843 at index: 17
the individual 6 id is equal to 3
Minimum value: 840.4098 at index: 22
the individual 7 id is equal to 4
Minimum value: 1587.2358 at index: 20
the individual 8 id is equal to 4
Minimum value: 1229.0576 at index: 28
the individual 9 id is equal to 5
Minimum value: 1249.7853 at index: 28
the individual 10 id is equal to 5
Minimum value: 474.7445 at index: 33
the individual 11 id is equal to 6
Minimum value: 703.1169 at index: 33
the individual 12 id is equal to 6
Minimum value: 440.9014 at index: 39
the individual 13 id is equal to 7
Minimum value: 2317.3746 at index: 12
the individual 14 id is equal to 2
Minimum value: 1158.6443 at index: 45
the individual 15 id is equal to 8
Minimum value: 2109.2910 at index: 19
the individual 16 id is equal to 4
Minimum value: 1200.8978 at index: 51
the individual 17 id is equal to 9
Minimum value: 1720.2141 at index: 51
the individual 18 id is equal to 9
Minimum value: 1925.1807 at index: 57
the individual 19 id is equal to 10
Minimum value: 1076.9313 at index: 56
the individual 20 id is equal to 10
Minimum value: 1438.0958 at index: 63
the individual 21 id is equal to 11
Minimum value: 1797.2148 at index: 66
the individual 22 id is equal to 11
Minimum value: 1508.4293 at index: 71
the individual 23 id is equal to 12
Minimum value: 1528.4237 at index: 35
the individual 24 id is equal to 6
Minimum value: 1319.7475 at index: 78
the individual 25 id is equal to 13
Minimum value: 1228.8391 at index: 74
the individual 26 id is equal to 13
Minimum value: 1192.9457 at index: 80
the individual 27 id is equal to 14
Minimum value: 1556.3160 at index: 82
the individual 28 id is equal to 14
Minimum value: 1565.7609 at index: 87
the individual 29 id is equal to 15
Minimum value: 2189.8092 at index: 89
the individual 30 id is equal to 15
Minimum value: 1391.7021 at index: 3
the individual 1 id is equal to 1
Minimum value: 1634.0244 at index: 2
the individual 2 id is equal to 1
Minimum value: 1338.1893 at index: 9
the individual 3 id is equal to 2
Minimum value: 2053.8975 at index: 8
the individual 4 id is equal to 2
Minimum value: 1540.0228 at index: 15
the individual 5 id is equal to 3
Minimum value: 2166.4656 at index: 17
the individual 6 id is equal to 3
Minimum value: 877.9372 at index: 22
the individual 7 id is equal to 4
Minimum value: 1626.0167 at index: 20
the individual 8 id is equal to 4
Minimum value: 1296.7123 at index: 28
the individual 9 id is equal to 5
Minimum value: 1299.9681 at index: 28

the individual 10 id is equal to 5
 Minimum value: 553.5969 at index: 33
 the individual 11 id is equal to 6
 Minimum value: 772.8253 at index: 33
 the individual 12 id is equal to 6
 Minimum value: 448.7949 at index: 39
 the individual 13 id is equal to 7
 Minimum value: 2334.9615 at index: 12
 the individual 14 id is equal to 2
 Minimum value: 1171.0828 at index: 45
 the individual 15 id is equal to 8
 Minimum value: 2121.3320 at index: 19
 the individual 16 id is equal to 4
 Minimum value: 1214.5061 at index: 51
 the individual 17 id is equal to 9
 Minimum value: 1735.6472 at index: 51
 the individual 18 id is equal to 9
 Minimum value: 1955.2812 at index: 57
 the individual 19 id is equal to 10
 Minimum value: 1150.5081 at index: 56
 the individual 20 id is equal to 10
 Minimum value: 1465.8858 at index: 63
 the individual 21 id is equal to 11
 Minimum value: 1825.7211 at index: 66
 the individual 22 id is equal to 11
 Minimum value: 1516.0239 at index: 71
 the individual 23 id is equal to 12
 Minimum value: 1562.3842 at index: 35
 the individual 24 id is equal to 6
 Minimum value: 1329.4430 at index: 78
 the individual 25 id is equal to 13
 Minimum value: 1243.0952 at index: 74
 the individual 26 id is equal to 13
 Minimum value: 1248.4402 at index: 80
 the individual 27 id is equal to 14
 Minimum value: 1603.7415 at index: 82
 the individual 28 id is equal to 14
 Minimum value: 1582.8101 at index: 87
 the individual 29 id is equal to 15
 Minimum value: 2206.8755 at index: 89
 the individual 30 id is equal to 15

For k = 5

Image n1 Train id 1 Image n2 Train id 2 Image n3 Train id 3 Image n4 Train id 4 Image n5 Train id 5



Image n6 Train id 6 Image n7 Train id 7 Image n8 Train id 8 Image n9 Train id 9 Image n10 Train id 10



Image n11 Train id 11 Image n12 Train id 12 Image n13 Train id 13 Image n14 Train id 14 Image n15 Train id 15



Image n16 Train id 16 Image n17 Train id 17 Image n18 Train id 18 Image n19 Train id 19 Image n20 Train id 20



Image n21 Train id 21 Image n22 Train id 22 Image n23 Train id 23 Image n24 Train id 24 Image n25 Train id 25



Image n26 Train id 26 Image n27 Train id 27 Image n28 Train id 28 Image n29 Train id 29 Image n30 Train id 30



For k = 30



For k = 50



For k = 75



For k = 90



Pour aller plus loin

```
k = 30;

%pour l'erreur minimale de la base d'entrainement pour k=30

erreur_train = zeros(1, 90);
xmoy = mean(X_train,2);
X_c = X_train - xmoy; %calcul image centré

for indice_x=1:90 %calculer l'erreur pour toutes les personnes
x = X_train(:,indice_x); %prenons comme exemple la premiere image
z = U(:,1:k)' * X_c(:,indice_x); %x-xmoyenne represente l'image centree de x
x_reconstruit = xmoy + U(:,1:k)*z; %l'image reconstruite de x

erreur_train(indice_x) = norm(x-x_reconstruit); %l'erreur entre l'image et on image reconstruite

end

max_erreur_train = max(erreur_train);
moy_erreur_train = mean(erreur_train);
fprintf('the maximale de la base train %d et la moyenne est égale à %d\n',max_erreur_train, moy_erreur_train);

figure(30);
subplot(1,2,1); % image origine
imshow(mat2gray(imageto64(x)));
title("Image 1");
subplot(1,2,2); % image reconstruite pour la dernier image de X_train par exemple
imshow(mat2gray(imageto64(x_reconstruit)));
title("Reconstruction train");

%*****
%pour l'erreur minimale de la base de test pour k=30
erreur_test = zeros(1, 30);
xmoy_test = mean(X_train,2);
X_c_test = X_test - xmoy_test; %calcul image centré
for indice_x=1:30 %calculer l'erreur pour toutes les personnes
x_test = X_test(:,indice_x); %prenons comme exemple la premiere image
z = U(:,1:k)' * X_c_test(:,indice_x); %x-xmoyenne represente l'image centree de x

x_reconstruit_test = xmoy_test + U(:,1:k)*z; %l'image reconstruite de x

erreur_test(indice_x) = norm(x_test-x_reconstruit_test); %l'erreur entre l'image et on image reconstruite

end
max_erreur_test = max(erreur_test);
moy_erreur_test = mean(erreur_test);
fprintf('the maximale de la base test %d et la moyenne est égale à %d\n',max_erreur_test, moy_erreur_test);

figure(31);
subplot(1,2,1); % image origine
imshow(mat2gray(imageto64(x_test)));
title("Image 1");
```

```

subplot(1,2,2); % image reconstruite pour la dernier image de X_test par exemple
imshow(mat2gray(imageto64(x_reconstruit_test)));
title("Reconstruction train");

%*****
%pour l'erreur minimale de la base no visage pour k=30
erreur_nf = zeros(1, 10);
xmoy_nf = mean(X_noface,2);
X_c_nf = X_noface - xmoy_nf; %calcul image centré
for indice_x=1:10 %calculer l'erreur pour toutes les personnes
x_nf = X_noface(:,indice_x); %prenons comme exemple la premiere image
z = U(:,1:k)' * X_c_nf(:,indice_x); %x-xmoyenne represente l'image centree de x

x_reconstruit_nf = xmoy_nf + U(:,1:k)*z; %l'image reconstruite de x

erreur_nf(indice_x) = norm(x_nf-x_reconstruit_nf); %l'erreur entre l'image et on image reconstruite

end
min_erreur_nf = min(erreur_nf); % calcul la valeur minimale
moy_erreur_nf = mean(erreur_nf);
fprintf('the minimum de la base no face %d et la moyenne est égale à %d\n',min_erreur_nf, moy_erreur_nf);

figure(32);
subplot(1,2,1); % image origine
imshow(mat2gray(imageto64(x_nf)));
title("Image 1");
subplot(1,2,2); % image reconstruite pour la dernier image de X_test par exemple
imshow(mat2gray(imageto64(x_reconstruit_nf)));
title("Reconstruction train");

%*****
%*****
% Algorithme de face/noface

disp("Voici les images qu'on doit classifier");
figure(33);
for i=1:10
    cat_noface{i} = mat2gray(imageto64(X_inconnu(:,i)));
    subplot(2,5, i);
    imshow(cat_noface{i});
    title("Image "+i);

end

erreur_face = mean([max_erreur_train,max_erreur_test,moy_erreur_test]) + 0.5*mean([moy_erreur_train,moy_erreur_test]);
% l'erreur moyenne d'un visage que ca soit entrainement ou train
%la formule que j'ai trouvé marche le plus
erreur_inc = zeros(1, 10);
xmoy_inc = mean(X_inconnu,2);
X_c_inc = X_inconnu - xmoy_inc; %calcul image centré
for indice_x=1:10 %calculer l'erreur pour toutes les personnes
x_inc = X_inconnu(:,indice_x); %prenons comme exemple la premiere image
z = U(:,1:k)' * X_c_inc(:,indice_x); %x-xmoyenne represente l'image centree de x

x_reconstruit_inc = xmoy_inc + U(:,1:k)*z; %l'image reconstruite de x

erreur_inc(indice_x) = norm(x_inc-x_reconstruit_inc); %l'erreur entre l'image et on image reconstruite
if erreur_inc(indice_x)<=erreur_face
    fprintf('the image avec le numero %d est un visage\n',indice_x);
else
    fprintf('the image avec le numero %d n est pas un visage\n',indice_x);
end

end
end

```

```

the maximale de la base train 1.080757e+03 et la moyenne est égale à 8.119745e+02
the maximale de la base test 1.738367e+03 et la moyenne est égale à 1.203129e+03
the minimum de la base no face 1.504219e+03 et la moyenne est égale à 2.600816e+03
Voici les images qu'on doit classifier
the image avec le numero 1 est un visage
the image avec le numero 2 est un visage
the image avec le numero 3 n est pas un visage
the image avec le numero 4 n est pas un visage
the image avec le numero 5 est un visage
the image avec le numero 6 est un visage
the image avec le numero 7 n est pas un visage
the image avec le numero 8 est un visage
the image avec le numero 9 n est pas un visage
the image avec le numero 10 est un visage

```


Image 1



Reconstruction train



Image 1



Reconstruction train



Image 1



Reconstruction train



Image 1



Image 2



Image 3



Image 4



Image 5



Image 6



Image 7



Image 8



Image 9



Image 10

